captain prospect.

WHITE-LABEL READINESS

One image, many agencies.

Plan, findings and execution sequence for runtime, per-instance branding — without touching the live Captain Prospect deployment.

PLAN ONLY · NO CODE OR DATABASE CHANGE YET

SCOPE	STEPS	DATE
CRM repository only	1 – 10	25 October 2026

Prepared for the engineering team · Control plane out of scope

CONTENTS

01 **What I verified in the repo**

02 **Assumptions in the brief that look wrong (or need a decision)**

03 **Target architecture**

04 **Data model (step 1)**

05 **Steps, files and acceptance checks**

06 **Verification plan (what I will actually run)**

07 **Commit sequence**

08 **Recommended defaults I will use unless told otherwise**

DOCUMENT CONTROL

Status	Plan for review. No schema, code or database change applied.
Scope	This CRM repository. The provisioning control plane is excluded.
Database	Live shared database. Not read, not migrated, not seeded.
Migration	New SQL drafted by hand and validated in PGlite only.
Checks	TypeScript, ESLint, brand:check, pure-unit tests, visual pass.
Deliverables	Phased code for steps 1–8 and 10; written proposal for step 9.

> **BEFORE CODE**
>
> Section 2 lists the assumptions in the brief that do not hold as written. Section 8 records the defaults this plan will use unless corrected.

01 What I verified in the repo

Fact	Evidence
Next.js 16.1.1, React 19.2.3, Prisma 6.19.1, Tailwind 4, output: "standalone"	package.json, next.config.ts:10
No BrandSettings, no IntegrationSecret, no tenantId anywhere	prisma/schema.prisma (124 models listed)
Client is the agency's customer, as stated	prisma/schema.prisma:698 and following
68 @/lib/brand import lines; 24 of the importing files are "use client"	grep from "@/lib/brand"
brand is a module-level const whose palette is computed once at import	lib/brand/index.ts:45, lib/brand/tokens.ts:19
BrandStyle is a synchronous server component reading the static palette	components/brand/BrandStyle.tsx:8
Root metadata, viewport and icons are static exports	app/layout.tsx:9-27
Middleware runs next-auth on every /api/* except auth, webhooks, v1, mcp	middleware.ts:62-75
No local Postgres, no Docker, no psql on this machine	docker --version, psql --version both not found
The root .env points at the live database	DATABASE_URL / DIRECT_URL present; values not read
Existing audit helper is fire-and-forget and requires a user actor	lib/audit.ts:55
Existing rate limiter is in-memory LRU, keyed by IP/email	lib/rate-limit.ts:11-33
MinIO helper already supports put/get/delete/presign	lib/storage/minio.ts:167-244
Runtime image has no prisma CLI, no prisma/ folder, no openssl, no GIT_SHA	Dockerfile:28-38
unstable_cache still exists in Next 16; revalidateTag now takes a profile argument	node_modules/next/cache.d.ts
next.config.ts sets typescript.ignoreBuildErrors: true	next.config.ts:18-20
The working tree already contains many unrelated uncommitted changes	git status

Reference-data and seed sources found:

- `prisma/seed.ts` — permissions, role permissions, global action statuses, result categories, plus demo client/SDR/manager/users/mission/list/companies.
- `prisma/seed-result-categories.ts`, `prisma/seed-endpoints.ts`.
- Root SQL: `add-statuses.sql`, `seed-admin.sql`, `seed-data.sql`, `seed-data-extended.sql`, `supabase-setup.sql`, `supabase-from-schema.sql` (82.8 KB), `supabase-full-schema.sql` (**0 KB — empty file**), `alter-enum.sql`, `performance-indexes.sql`, plus `migrate-missions-to-global.sql`.

02 Assumptions in the brief that look wrong (or need a decision)

A1. "Keep the sync brand export working" is only half-true

A client component cannot read the database, and `NEXT_PUBLIC_*` values are inlined at build time. So 24 client components can never see a runtime brand through the current import.

Plan: keep `brand` as the *default* brand (same type, same helpers) so all 68 imports keep compiling, and add:

- `getBrand()` on the server (DB + cache, async),
- `<BrandProvider>` in the root layout + `useBrand()` in client components.

The 24 client files are migrated to `useBrand()` (or receive values as props). Anything left on `brand` renders Captain Prospect values — that is a bug we can grep for. "Keeps working where possible" = compiles everywhere, runtime values only where the component can receive them.

A2. "Per request" in the root layout makes the whole app dynamic

Reading the DB in `app/layout.tsx` (metadata included) removes static prerendering for every route: `/login`, `/design-system`, shared report pages, etc. become server-rendered per request. That is the price of runtime branding and is acceptable for an auth-heavy CRM, but it must be a conscious decision.

Two sub-problems:

1. `next build` in Docker has no database. `getBrand()` must **never throw**: it falls back to `brandConfig` and logs. Otherwise the image cannot build.
2. Without an explicit dynamic API, Next would prerender pages at build time with the fallback brand and serve that snapshot. The layout must call `await headers()` (or `connection()`) so every request resolves the brand.

Decision proposed: accept dynamic rendering, add the explicit dynamic call, and document the trade-off. Cached DB read (60 s TTL + tag) keeps the cost to one small query per minute, not per request.

A3. NEXT_PUBLIC_* on the server is not frozen

Only client bundles inline `NEXT_PUBLIC_*`. Server routes reading `process.env.NEXT_PUBLIC_APP_URL` already get the runtime value. The real runtime breakage is on the client side:

Variable	Where it actually breaks	Move to
NEXT_PUBLIC_QONTO_ENABLED, NEXT_PUBLIC_PAPPERS_ENABLED	app/manager/billing/settings/page.tsx:391,396 (client)	runtime config injected by the layout / /api/public-config
NEXT_PUBLIC_SOCKET_URL, NEXT_PUBLIC_SOCKET_PATH	lib/socket/comms-socket.ts (client)	same
NEXT_PUBLIC_OPENREPLAY_*	lib/openreplay/tracker.ts (client)	same
NEXT_PUBLIC_APP_URL	components/prospects/SourceSetupWizard.tsx:429 (client), app/manager/clients/[id]/page.tsx:844 (client)	server prop or runtime config
NEXT_PUBLIC_SUPABASE_URL	server-only (lib/storage/supabase-report-storage.ts) + scripts	rename to SUPABASE_URL with fallback; no public exposure

Server-side uses (`lib/slack/clientsLive.ts`, `lib/integrations/messaging/*`, Slack OAuth, API routes) are moved to `brand.appUrl` / `getBrand()` for one source of truth, not because they are broken.

A4. Runtime font choice is impossible with next/font

`brand/fonts.ts` uses literal `next/font/google` calls at module scope; fonts are downloaded and hashed at build time. A DB value cannot add a family at runtime.

Proposal: preload a small fixed set (3 presets: current Bricolage/Geist, Inter/Inter, Manrope/Manrope) at build time and let `BrandSettings.fontPreset` switch the `--font-display-face` / `--font-body-face` variables. Document that adding a new family still needs a rebuild.

A5. npm run db:migrate:deploy cannot run in the current runtime image

The runtime stage copies only `.next/standalone`, `.next/static` and `public`. It has no `prisma` CLI and no `prisma/migrations`. A one-shot command in that image would fail.

Proposal: add a dedicated `migrate` build stage (`FROM deps`, copy `prisma/`, `CMD ["npx","prisma","migrate","deploy"]`) and the npm script `db:migrate:deploy`. The control plane runs that stage as a one-shot container before switching traffic. The runtime image stays migration-free.

A6. Bootstrap "one-time password" has no field to enforce it

`User` has no `mustChangePassword`. The existing `PasswordResetToken` table can carry an invite instead.

Proposal: `POST /api/instance/bootstrap` creates the first `MANAGER` with a random unusable password plus a `PasswordResetToken` (24 h), and returns the invite URL. No schema change, and the password is never sent in clear. Alternative if you prefer a temporary password: add `mustChangePassword` to `User` — one extra column.

A7. I cannot apply or verify the migration here

No Docker, no Postgres, no `psql` on this machine, and the `.env` database is live. I will:

- write the migration SQL by hand (one table, additive),
- validate it by applying it to PGlite,
- run `prisma validate` and `prisma generate`,
- **not** run `prisma migrate deploy` anywhere.

A real `migrate deploy` run on a throwaway Postgres is still required before calling the migration production-ready.

A8. Per-step commits will not be clean

The tree already has modified and deleted files unrelated to this job. Committing `git add -A` per step would sweep them in. I will stage **only the paths I touch** in each step and say so in the commit body.

A9. CSP is build-time, and it hardcodes the OpenReplay host

`next.config.ts:57` allows `https://openreplay.suzaliconseil.com`. `headers()` is evaluated at build, so moving the ingest URL to a DB setting cannot change the CSP at runtime.

Proposal: read the ingest origin from `OPENREPLAY_INGEST_ORIGIN` (build arg) and keep the client tracker runtime-configured; document that a per-instance CSP host needs either a build arg or a middleware-generated header. I will not invent a middleware CSP in this pass (it would run on the edge and risks breaking other headers).

A10. ListType.SUZALI cannot be renamed at runtime

It is a Postgres enum value used by existing rows and code. Renaming it is a data migration with a deploy-order problem on the live DB. Keep it as a legacy enum value; the *label* shown to users comes from `brand.companyShortName`.

A11. Emails and PDFs are synchronous and capture brand at call time

`lib/email/templates/*`, `lib/reporting/pdf.ts`, `lib/billing/facturx-service.ts` build strings synchronously. Turning them into `await getBrand()` cascades into call sites (some inside non-async helpers).

Proposal: thread an optional `Brand` parameter (`render(brand, data)`) and resolve once at the route/worker boundary; keep a default argument for backwards compatibility. This avoids making the whole call graph async.

A12. Logo uploads have no fallback storage

The container has no writable persistent disk and the runtime image only copies `public/`. If `MINIO_*` is missing, uploads must fail cleanly (503) and the UI must keep the `/public/brand/*` defaults. There is no local-disk fallback.

A13. seed-admin.sql contains a real credential

`seed-admin.sql` embeds a bcrypt hash and documents the password in a comment (`admin@captain-prospect.fr` / `Captain123@`). This should be treated as a leaked credential: rotate it, and replace the file with a non-secret placeholder (the file is kept, not deleted). Flagging for your action — I will not run it.

03 Target architecture

```
brand/brand.config.ts        defaults, unchanged shape (built once, client-safe)
        │
lib/brand/types.ts           Brand (resolved), BrandSettingsInput, BrandAssetSlot
lib/brand/resolve.ts         pure: defaults + DB row → Brand (palette via color.ts)
lib/brand/index.ts           brand = resolve(defaults) + brandUrl/brandSender/…
lib/brand/server.ts          getBrand() cached · getBrandUncached() · invalidateBrand()
lib/brand/client.tsx         <BrandProvider brand> + useBrand()
lib/brand/tokens.ts          brandCss(brand) — pure, no import of the singleton
        │
app/layout.tsx (async)       getBrand() + getRuntimeConfig()
  ├─ <BrandStyle brand>      :root CSS vars
  ├─ <BrandProvider brand>   client tree
  ├─ <RuntimeConfig>         socket / OpenReplay / feature flags / app URL
  └─ generateMetadata()      title, description, icons, theme colour
```

Caching:

- `unstable_cache` key `["brand-settings"]`, tag `"brand"`, `revalidate: 60`.
- `cache()` from React to dedupe within one request.
- Writes call `revalidateTag("brand", { expire: 0 })` and `revalidatePath("/", "layout")` (Next 16 signature).
- DB unavailable → defaults + `console.warn`, never throw.
- Workers, cron scripts and PDF jobs use `getBrandUncached()` (no Next cache context outside the app runtime).

Middleware stays database-free. `/api/brand/assets/*`, `/api/public-config` and `/api/instance/*` are excluded from the `next-auth` matcher and protect themselves.

04 Data model (step 1)

`BrandSettings` — exactly one row, `id = 'default'`:

```
id                  String   @id @default("default")
name                String   @default("Captain Prospect")
productName         String   @default("Captain Prospect CRM")
tagline             String
description         String
companyName         String
companyShortName    String

appUrl              String
website             String

senderName          String
notificationsAddress String
supportAddress      String
emailDomains        String[]

lang                String   @default("fr")
locale              String   @default("fr-FR")
currency            String   @default("EUR")
timeZone            String   @default("Europe/Paris")

primaryColor        String   @default("#263460")
accentColor         String   @default("#C64B8B")
neutralTint         Float    @default(1)

fontPreset          String   @default("default")

logoFullKey         String?
logoFullInverseKey  String?
logoMarkKey         String?
logoMarkInverseKey  String?
faviconKey          String?
appleTouchIconKey   String?
logoFullWidth       Int      @default(2460)
logoFullHeight      Int      @default(758)
logoVersion         String?

updatedAt           DateTime @updatedAt
updatedBy           String?
```

- Every default is the current `brandConfig` value, so an empty table renders exactly today's Captain Prospect.
- The migration seeds the `default` row (idempotent) so `getBrand()` has a row; the code still tolerates an empty table.
- `logoVersion` is bumped on upload and used as the `?v=` cache-buster.
- No `tenantId`, no relation to `Client`.

Step 9 proposal (document only, no table created now): `IntegrationSecret` (`key` unique, `ciphertext`, `keyVersion`, `updatedBy`, `updatedAt`) encrypted with `lib/encryption.ts`, manager UI, env fallback, audit on read/write.

05 Steps, files and acceptance checks

Step 1 — schema + migration

Files: `prisma/schema.prisma`, `prisma/migrations/20261025120000_add_brand_settings/migration.sql`.

Checks: `npx prisma validate`, `npx prisma generate`, apply SQL in PGlite, `npx tsc --noEmit`. Commit: `feat(brand): add BrandSettings model and migration`.

Step 2 — runtime brand resolution

Files: `lib/brand/{types,resolve,server,client,tokens,index}.ts`, `components/brand/{BrandStyle,BrandLogo}.tsx`, `brand/fonts.ts`, `brand/README.md`, plus the 68 import sites (server first, then the 24 client files).

Checks: `npm run brand:check`, `npx tsc --noEmit`, `npm run lint`, no client file imports `lib/brand/server`.

Step 3 — layout, metadata, runtime config

Files: `app/layout.tsx`, `lib/runtime-config.ts`, `components/runtime/RuntimeConfig.tsx`, `app/api/public-config/route.ts`, the `NEXT_PUBLIC_*` call sites listed in A3, `next.config.ts` (CSP host).

Checks: metadata and icons carry `?v=`; no `NEXT_PUBLIC_` left in client code except the documented build-time CSP origin; public config contains no secret.

Step 4 — logo and favicon upload/serve

Files: `lib/brand/assets.ts` (validation, PNG/WebP dimensions, SVG sanitizer), `lib/brand/upload.ts` (MinIO, prefix `brand/`), `app/api/brand/assets/[slot]/route.ts`, `app/api/manager/brand/assets/route.ts`. Route: `GET /api/brand/assets/:slot?v=…` → `Cache-Control: public, max-age=31536000, immutable`, `ETag`, correct `Content-Type`, `X-Content-Type-Options: nosniff`; 404 → redirect to the `/public/brand/*` default. Validation: png/webp/svg, ≤ 2 MB, ≤ 4096 px, SVG sanitized (no script, foreignObject, event handlers, external refs). Unit tests with `node --test`.

Step 5 — manager branding UI

Files: `app/manager/settings/page.tsx` (new "Identité" section), `app/manager/settings/BrandingPanel.tsx`, `app/api/manager/brand/route.ts` (GET/PUT), `app/api/manager/brand/reset/route.ts`, `lib/audit.ts` (new `BRAND_UPDATE` action; `auditSystem()` for the API). Role check: middleware + `requireRole(["MANAGER"])`. Live preview reuses `lib/brand/color.ts` and the contrast helpers from `/design-system`. Every field change is audit-logged with before/after; reset is audited too.

Step 6 — provisioning API

Files: `lib/instance/auth.ts` (HMAC), `lib/instance/rate-limit.ts`, `app/api/instance/brand/route.ts`, `app/api/instance/brand/assets/route.ts`, `app/api/instance/health/route.ts`, `app/api/instance/bootstrap/route.ts`.

Signature:

```
X-Instance-Timestamp: 1760000000
X-Instance-Signature: v1=<hex hmac-sha256>
payload = `${timestamp}.${METHOD}.${pathWithQuery}.${sha256(rawBody)}`
```

- `timestamp` in seconds, reject if `|now - ts| > 300`.
- `crypto.timingSafeEqual` on equal-length buffers; reject unknown `v1`.
- Secret `INSTANCE_ADMIN_SECRET`; missing secret → 503, never a default key.
- Rate limit per IP + route (in-memory LRU, 30/min).
- `GET /api/instance/health` → `{ version, schemaVersion, brandUpdatedAt, dbOk }`; `schemaVersion` from `_prisma_migrations` via raw SQL, `null` if absent.
- `POST /api/instance/bootstrap` → 409 if any user exists; creates MANAGER + invite token (A6); idempotent on retry only while no user exists.

Step 7 — instance seed

Files: `prisma/seed-instance.ts`, `package.json` (`db:seed:instance`), docs. Content: BrandSettings row, permissions and role permissions (extracted from `prisma/seed.ts`), global action statuses, result categories, `SystemConfig` defaults, no demo users, no prospect data, no secrets. `prisma/seed.ts` keeps its demo data for local development and imports the shared reference-data module so the two cannot drift. Root `.sql` files stay; a mapping table in `prisma/README` says which are historical and which are superseded.

Step 8 — hard-coded brand strings

Current scan: 110 hits in 45 files (including docs, `brand.config.ts`, manual SQL). Code to fix first: `app/page.tsx` ("Suzalink"), `next.config.ts` (CSP host), `lib/openreplay/tracker.ts`, `lib/types.ts` / `ListType` label, `lib/prospection-export/*`, `lib/vault/*`, `lib/support/service.ts`, `prisma/seed.ts`, root `seed-admin.sql`. Deliverable: `docs/BRAND_STRINGS_REPORT.md` listing what changed and what stays (protocol values, legacy enum, migration files, historical docs).

Step 9 — per-instance secrets (proposal only)

Files: `docs/INTEGRATION_SECRETS_PROPOSAL.md`. Content: inventory of every key read in the repo (Allo, Leexi, Qonto, Mistral, OpenAI, Gemini, Pappers, Microsoft, Slack, Supabase, SMTP, MinIO, Vault, ReachInbox, Cognism, Serp/Serper, Google Places), where it is read, whether it is global or per-instance today; `ENCRYPTION_KEY` / `NEXTAUTH_SECRET` analysis; `IntegrationSecret` schema, encryption and key-versioning, env fallback order, rotation and audit; migration path. No code.

Step 10 — Docker, migrations, health

Files: `Dockerfile`, `package.json`, `.dockerignore`, `docs/WHITE_LABEL.md`.

- `apt-get install openssl ca-certificates` in deps, build and runtime (fixes the Prisma libssl warning on `node:20-slim`).
- `ARG GIT_SHA=dev` → `ENV GIT_SHA` in build and runtime.
- `migrate` stage (A5) + `npm run db:migrate:deploy`.
- Runtime still starts with `node server.js`, never migrates.
- `.dockerignore` must keep `.env`, `.next`, `node_modules` out of the context.

Docs: `docs/WHITE_LABEL.md` (data model, endpoints + signature examples, seed and migration commands, required env vars) and an updated `brand/README.md`.

06 Verification plan (what I will actually run)

- `npx tsc --noEmit` (the build ignores type errors, so this is the real gate).
- `npm run lint`.
- `npm run brand:check`.
- `node --test` for the new pure units (SVG sanitizer, image dimensions, HMAC canonical string, resolve/merge, cache fallback).
- PGlite for the new migration SQL.
- Visual: `/login`, sidebar, `/design-system`, one email template, one PDF. These need a reachable database. With none available locally, I can render `/login` and `/design-system` against the fallback brand, but the sidebar, email and PDF checks require a throwaway Postgres. I will report exactly which of the five were rendered and which were not, rather than claim them.
- No migration, seed or write against the live `DATABASE_URL`.

07 Commit sequence

One commit per numbered step, staged by explicit path (A8):

1. `feat(brand): add BrandSettings model and migration`
2. `refactor(brand): resolve brand at runtime with cached getBrand`
3. `feat(brand): per-request metadata, CSS tokens and runtime public config`
4. `feat(brand): MinIO-backed logo and favicon assets`
5. `feat(manager): branding settings panel with audit trail`
6. `feat(instance): signed provisioning API, health and bootstrap`
7. `chore(seed): idempotent instance seed for fresh databases`
8. `refactor(brand): remove hard-coded agency strings`
9. `docs(brand): per-instance secrets proposal`
10. `build(docker): GIT_SHA, openssl and one-shot migrate stage`

Each commit is only made after its checks pass.

08 Recommended defaults I will use unless told otherwise

1. Dynamic rendering for the whole app (A2), cached brand read with 60 s TTL.
2. Invite token for bootstrap, no `mustChangePassword` column (A6).
3. Three preloaded font presets (A4).
4. Logo slots as explicit columns, SVG allowed but sanitized, 2 MB / 4096 px.
5. In-memory rate limiting for the instance API (single container per instance).
6. `/api/instance/health` requires the same HMAC signature as the other instance routes; only `/api/brand/assets/*` and `/api/public-config` are public.
7. `IntegrationSecret` stays a written proposal in this pass (as asked).